

ENGENHARIA DE SOFTWARE II

Prof. Júlio Machado

MICROSSERVIÇOS

Introdução

Decomposição de Sistemas

- Saber como decompor um sistema em componentes é uma das principais tarefas de um arquiteto de software
- Os componentes:
 - Podem ser desenvolvidos em paralelo
 - Podem ser reusados e/ou substituídos se a tecnologia sob a qual foram construídos mudar
 - Podem ser distribuídos em diferentes computadores

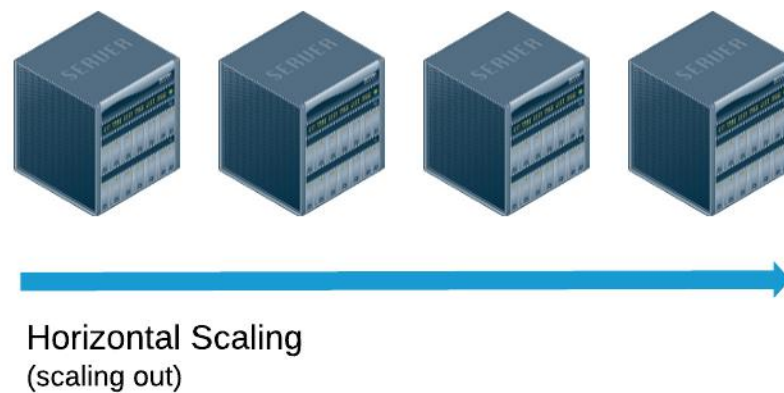
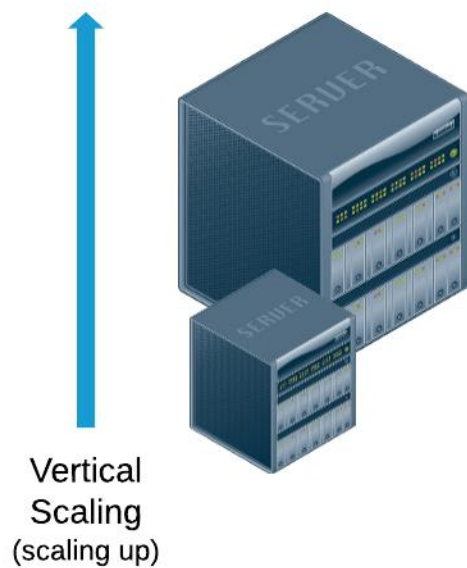
Sistemas na Nuvem

- Entre as vantagens de software baseado em nuvem podemos citar:
 - Escalabilidade
 - Confiabilidade
 - Elasticidade (gestão dos recursos)
- Para que se possa usufruir dessas vantagens são necessários componentes que possam ser:
 - Replicados
 - Executados em paralelo
 - Movidos de um servidor para outro

Escalabilidade

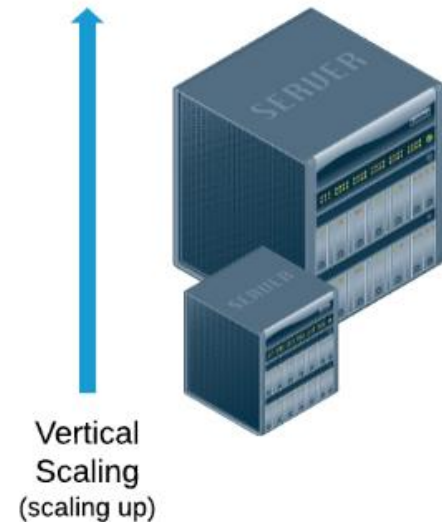
- A **escalabilidade** na computação em nuvem é a capacidade de aumentar ou diminuir de modo rápido e fácil o tamanho ou a potência de uma solução de TI ou de um recurso.
- Um sistema é **escalável** se possui a capacidade de gerenciar uma quantidade cada vez maior de trabalho.
- Dividida em duas abordagens:
 - Escalabilidade horizontal
 - Escalabilidade vertical

Escalabilidad



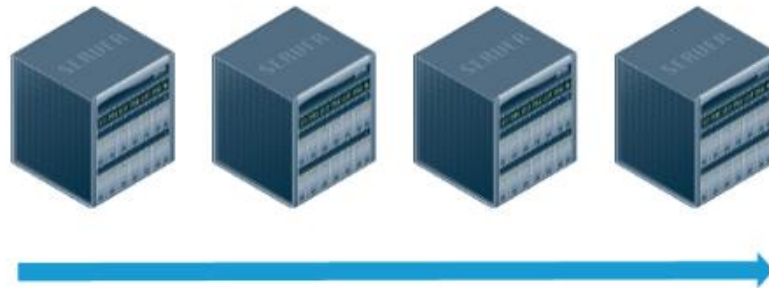
Escalabilidade Vertical

- O escalonamento vertical, também conhecido como *scale up/down*, é uma técnica em que a capacidade de um sistema adiciona (remove) poder de processamento e memória aos nós individuais à medida que a carga de trabalho aumenta (diminui).



Escalabilidade Horizontal

- O escalonamento horizontal, também conhecido como *scale out/in*, é uma técnica em que a capacidade de um sistema adiciona (remove) novos nós de processamento à medida que a carga de trabalho aumenta (diminui).



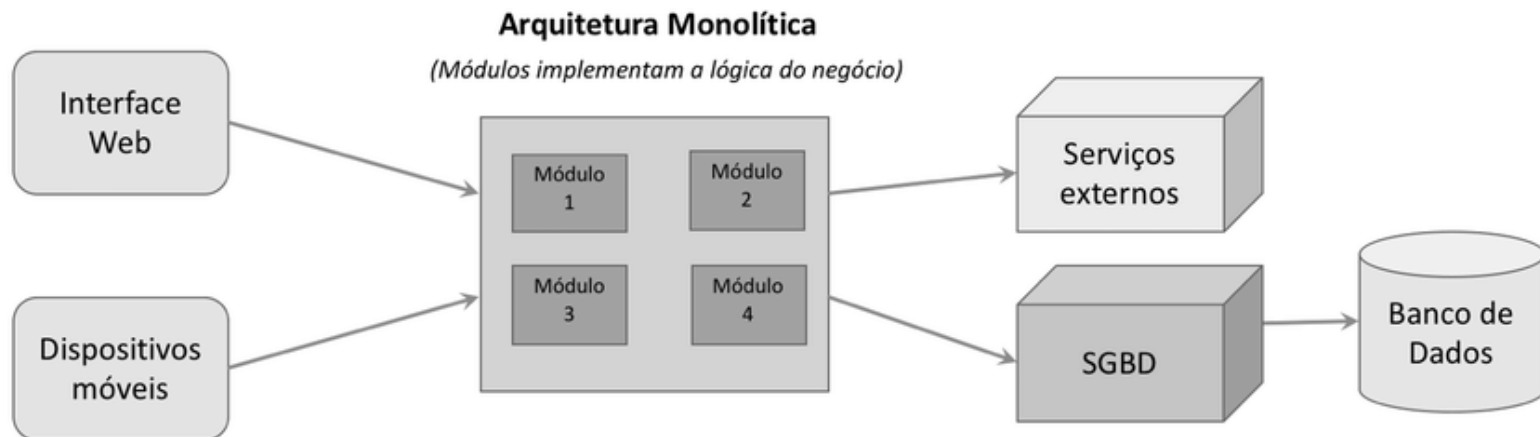
Horizontal Scaling
(scaling out)

Monolitos

Aplicação contém toda a lógica de negócio, implementada usualmente através de módulos.

Apesar de dividida em módulos, é implantada como um único bloco.

Monolitos



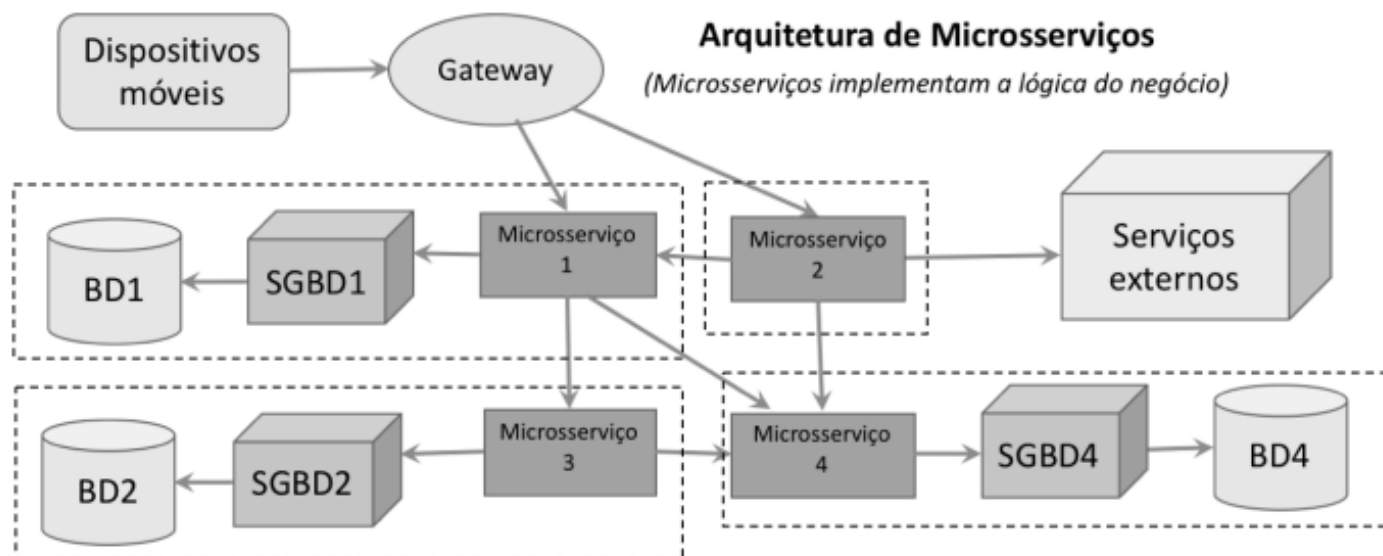
Adaptado de <https://www.nginx.com/blog/introduction-to-microservices/>

Microserviços

Microserviços emergiram da evolução nas formas de desenvolver sistemas.

Emergiram como uma tendência ou um padrão a partir do uso real.

Microserviços



Adaptado de <https://www.nginx.com/blog/introduction-to-microservices/>

Microserviços

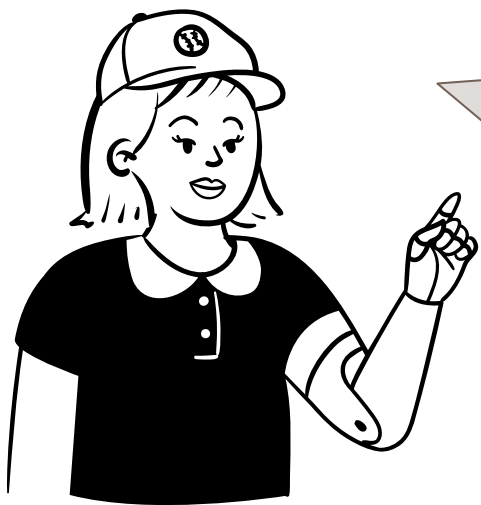
In short, the microservice architectural style is an approach to developing a single application as a **suite of small services**, each **running in its own process** and communicating with lightweight mechanisms, often an HTTP resource API. These services are **built around business capabilities** and **independently deployable** by fully automated deployment machinery. There is a **bare minimum of centralized management** of these services, which may be written in different programming languages and use different data storage technologies.

-- James Lewis and Martin Fowler (2014)

Microserviços

- Uma arquitetura de microserviços se constitui:
 - em uma coleção de pequenos serviços autônomos autocontidos,
 - de granularidade fina,
 - e que implementam uma única peça de responsabilidade de negócio.

Microserviços



Não é uma questão de tamanho, é uma questão de escopo!

Características

Pequenos,
independentes e
fracamente
acoplados

Mantidos em
bases-de-código
separadas

Implantados de
forma
independente de
outros serviços

Características



Autocontido

Os microsserviços não têm dependências externas. Eles gerenciam seus próprios dados e implementam sua própria interface de comunicação.



Leve

Os microsserviços se comunicam usando protocolos leves, para que as sobrecargas de comunicação de serviço sejam baixas.



Independente de implementação

Os microsserviços podem ser implementados usando diferentes linguagens de programação e podem usar diferentes tecnologias (por exemplo, diferentes tipos de banco de dados) em sua implementação.



Implantável de forma independente

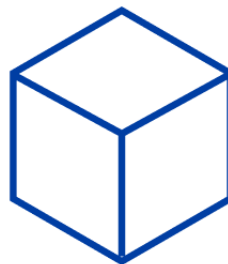
Cada microsserviço é executado em seu próprio processo e pode ser implantado de forma independente, usando sistemas automatizados.



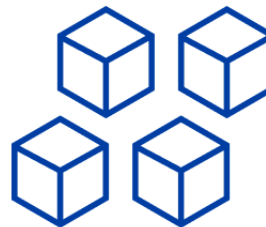
Orientado a negócios

Os microsserviços devem implementar recursos e necessidades de negócios, em vez de simplesmente fornecer um serviço técnico.

Comparação de modelos



MONOLITHIC
Single unit



SOA
Coarse-grained



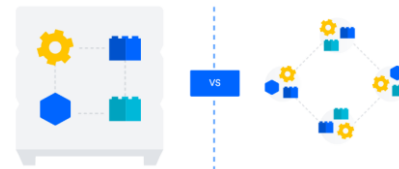
MICROSERVICES
Fine-grained

Monolitos vs Microserviços



Categoria	Arquitetura Monolítica	Arquitetura de Microserviços
Projeto	Base de código única com várias funções interdependentes.	Componentes de software independentes com funcionalidade autônoma que se comunicam entre si usando APIs.
Desenvolvimento	Requer menos planejamento no início, mas fica cada vez mais complexo de entender e manter.	Requer mais planejamento e infraestrutura no início, mas fica mais fácil de gerenciar e manter com o tempo.
Implantação	Aplicação inteira implantada como uma única entidade.	Cada microserviço é uma entidade de software independente que requer implantação individual em contêineres.
Depuração	Rastreie o caminho do código no mesmo ambiente.	Requer ferramentas avançadas de depuração para rastrear a troca de dados entre vários microserviços.

Monolitos vs Microserviços



Categoria	Arquitetura Monolítica	Arquitetura de Microserviços
Modificação	Pequenas mudanças introduzem riscos maiores, pois afetam toda a base de código.	Você pode modificar microserviços individuais sem afetar toda a aplicação.
Escala	Você precisa escalar toda a aplicação, mesmo que apenas determinadas áreas funcionais tenham um aumento na demanda.	Você pode escalar microserviços individuais conforme necessário, o que economiza custos gerais de escalabilidade.
Investimento	Baixo investimento inicial à custa de maiores esforços contínuos e de manutenção.	Investimento adicional de tempo e custo para configurar a infraestrutura necessária e desenvolver a competência da equipe. No entanto, economia de custos, manutenção e adaptabilidade a longo prazo.

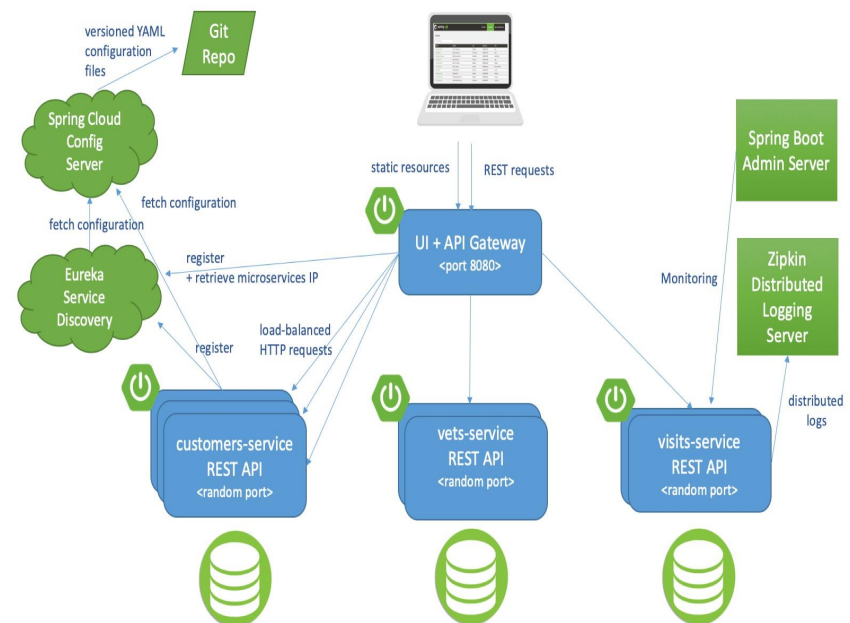
Exemplo: Petclinic

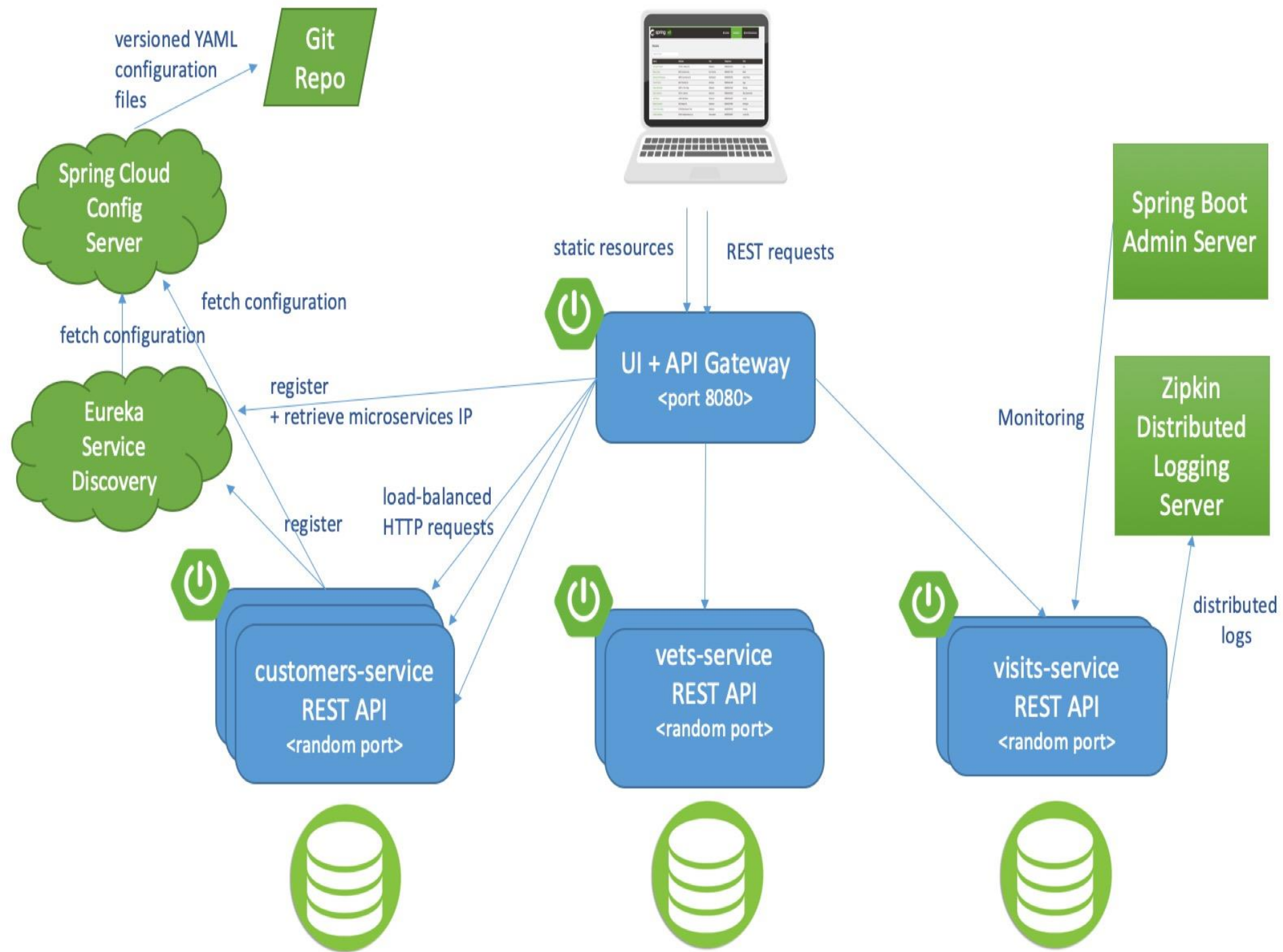
- Monolito

- <https://github.com/spring-projects/spring-petclinic>
- <https://github.com/spring-petclinic/spring-petclinic-rest>

- Microsserviços

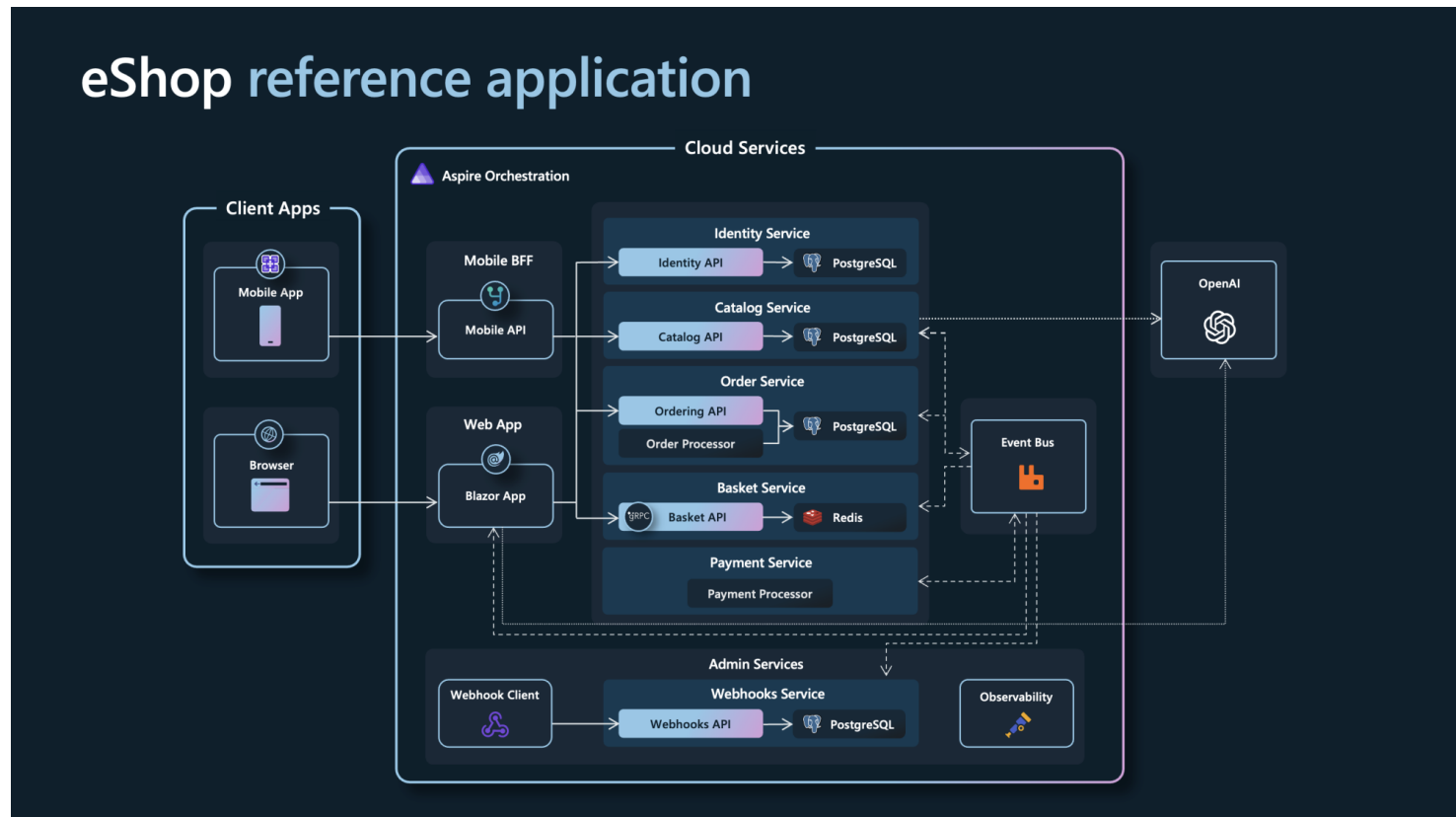
- <https://github.com/spring-petclinic/spring-petclinic-microservices>



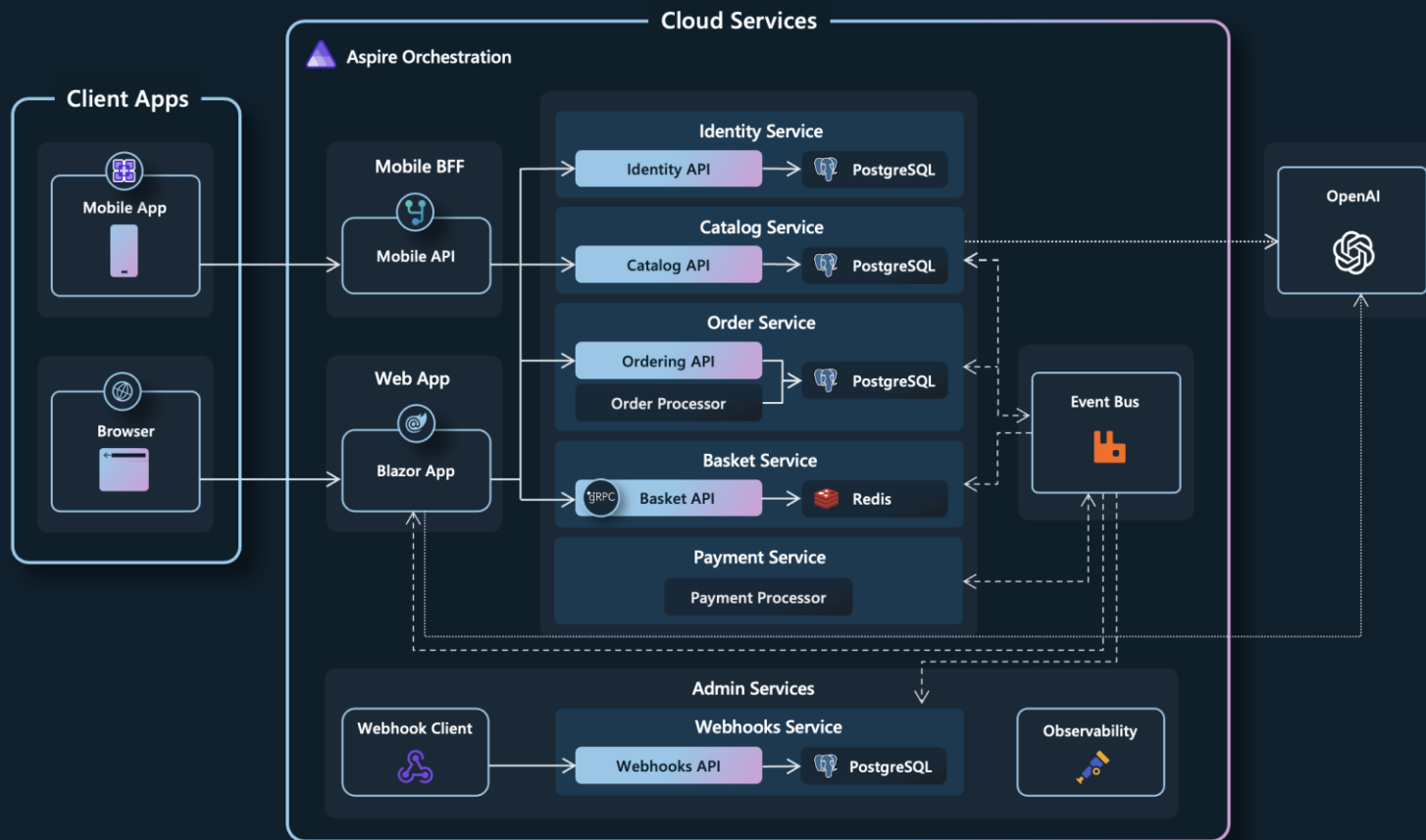


Exemplo: eShop

- <https://github.com/dotnet/eShop>



eShop reference application



Vantagens e desvantagens

- Baixo acoplamento
- Modularidade
- Falha em um serviço não impacta os demais
- Alta flexibilidade
- Alto grau de escalabilidade
- Facilidade de modificação
- Grande chance de falha na comunicação entre os serviços
- Dificuldade de gerenciar grande número de serviços
- Necessidade de resolver problemas tais como latência de rede, balanceamento de carga e outros problemas de arquiteturas distribuídas
- Teste mais complexo sobre um ambiente distribuído
- Implementação requer mais tempo

MICROSSERVIÇOS

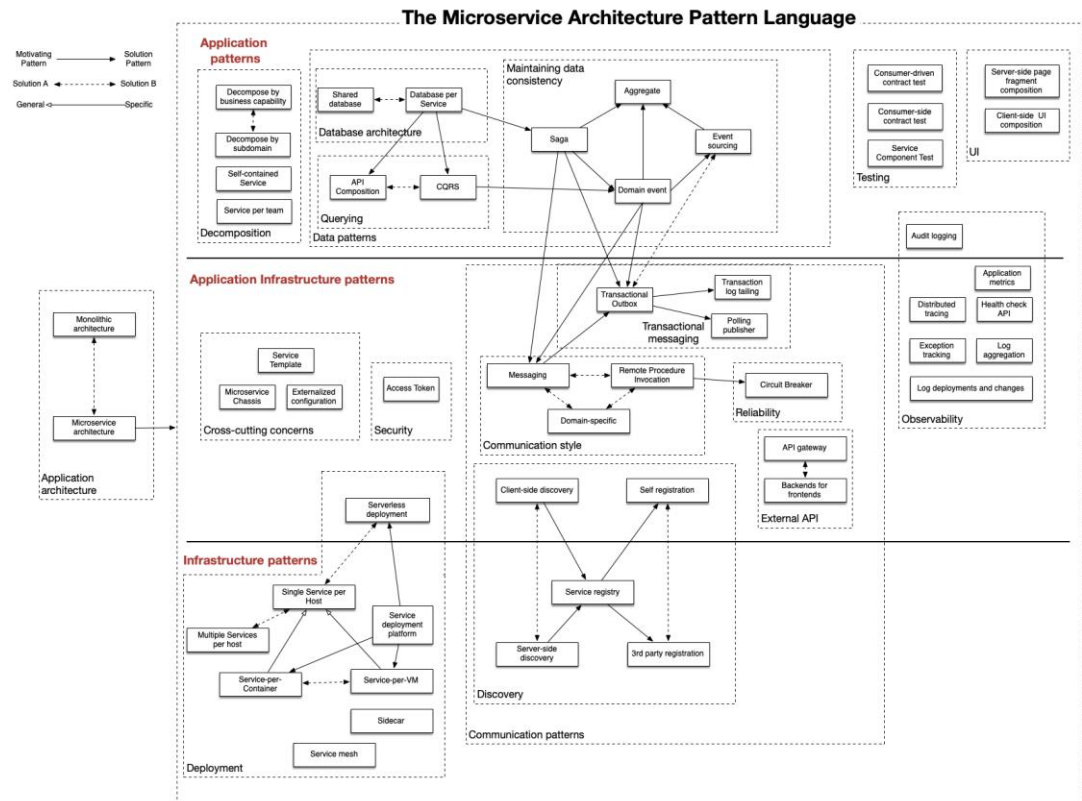
Diretrizes

Decisões Arquiteturais

- 
- Quais são os microsserviços que compõem o sistema?
 - Como os microsserviços se comunicam entre si?
 - Como os dados devem ser distribuídos e compartilhados?
 - Como os microsserviços devem ser coordenados?
 - Como as falhas nos microsserviços devem ser detectadas, reportadas e gerenciadas?

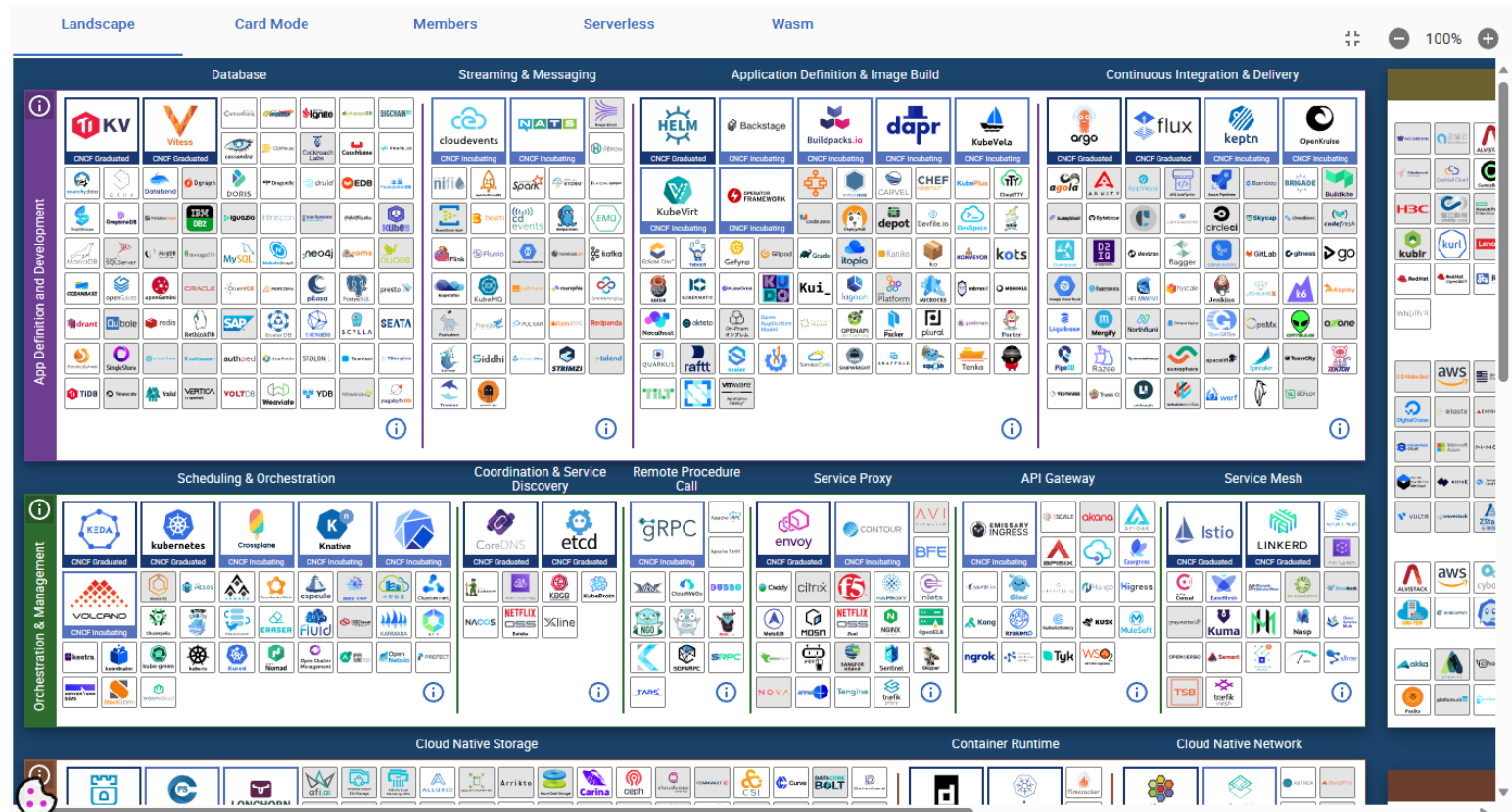
Decisões Arquiteturais

- Catálogos de padrões:
 - <https://microservices.io/>



Decisões Arquiteturais

- Tecnologias:
 - <https://landscape.cncf.io/>



MICROSSERVIÇOS

Decomposição

Decisões Arquiteturais



Quais são os microsserviços que compõem o sistema?



Como os microsserviços se comunicam entre si?



Como os dados devem ser distribuídos e compartilhados?



Como os microsserviços devem ser coordenados?



Como as falhas nos microsserviços devem ser detectadas, reportadas e gerenciadas?

Características Gerais

Um microserviço bem projetado deve ter alta coesão e baixo acoplamento.

- A coesão é uma medida do número de relacionamentos que as partes de um componente têm entre si. Alta coesão significa que todas as partes necessárias para fornecer a funcionalidade do componente estão incluídas no componente.
- O acoplamento é uma medida do número de relacionamentos que um componente tem com outros componentes do sistema. Baixo acoplamento significa que os componentes não têm muitos relacionamentos com outros componentes.

Características Gerais

Cada microserviço deve ter uma única responsabilidade, ou seja, deve fazer apenas uma coisa e deve fazê-lo bem.

- No entanto, 'uma coisa só' é difícil de definir de uma forma que seja aplicável a todos os serviços.
- A responsabilidade nem sempre significa uma atividade única e funcional.

Diretrizes de Decomposição



Equilibre a funcionalidade de grão fino e o desempenho do sistema

Os serviços de função única significam que as alterações são limitadas a menos serviços, mas exigem comunicações de serviço para implementar a funcionalidade do usuário. Isso torna o sistema mais lento devido à necessidade de cada serviço agrupar e desagrupar mensagens enviadas de outros serviços.



Siga o 'princípio de fechamento comum'

Elementos de um sistema que provavelmente serão alterados ao mesmo tempo devem estar localizados dentro do mesmo serviço. A maioria dos requisitos novos e alterados deve, portanto, afetar apenas um único serviço.



Associar serviços a recursos de negócios

Uma capacidade de negócios é uma área discreta de funcionalidade de negócios que é responsabilidade de um indivíduo ou de um grupo. Você deve identificar os serviços necessários para dar suporte a cada recurso de negócios.



Projete serviços para que eles tenham acesso apenas aos dados de que precisam

Se houver uma sobreposição entre os dados usados por diferentes serviços, você precisará de um mecanismo para propagar as alterações de dados para todos os serviços usando os mesmos dados.

Exemplo

- Autenticação do sistema:
 - Registro do usuário, onde os usuários fornecem informações sobre sua identidade, informações de segurança, número de telefone celular (celular) e endereço de e-mail
 - Autenticação usando UID/senha
 - Autenticação de dois fatores usando código enviado para celular
 - Gerenciamento de informações do usuário, por exemplo, alteração de senha ou número de celular
 - Redefinir senha esquecida

Exemplo

- Cada um desses recursos pode ser implementado como um serviço separado que usa um banco de dados central compartilhado para armazenar informações de autenticação.
- No entanto, esses recursos são muito grandes para serem microsserviços.
- Para identificar os microsserviços que podem ser usados no sistema de autenticação, você precisa dividir os recursos de granularidade grosseira em funções mais detalhadas.

Exemplo

- Detalhamento funcional dos recursos de autenticação:

User registration

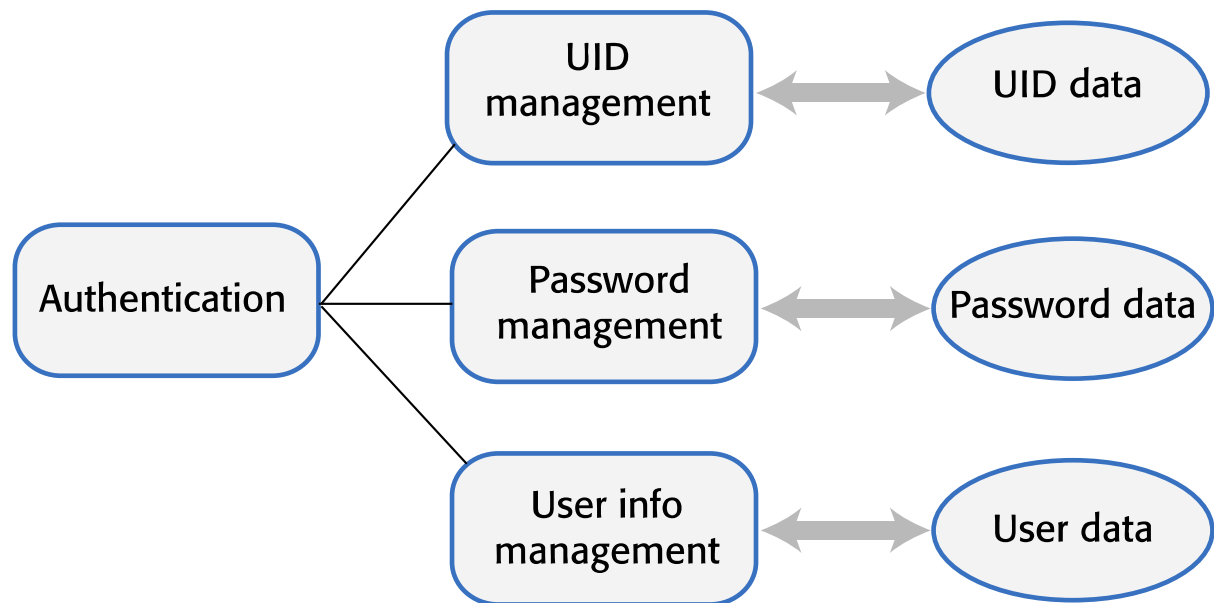
Setup new login id
Setup new password
Setup password recovery information
Setup two-factor authentication
Confirm registration

Authenticate using UID/password

Get login id
Get password
Check credentials
Confirm authentication

Exemplo

- Microserviços de autenticação:



Exemplo

- Funcionalidade de gerenciamento de senha:

User functions

Create password
Change password
Check password
Recover password

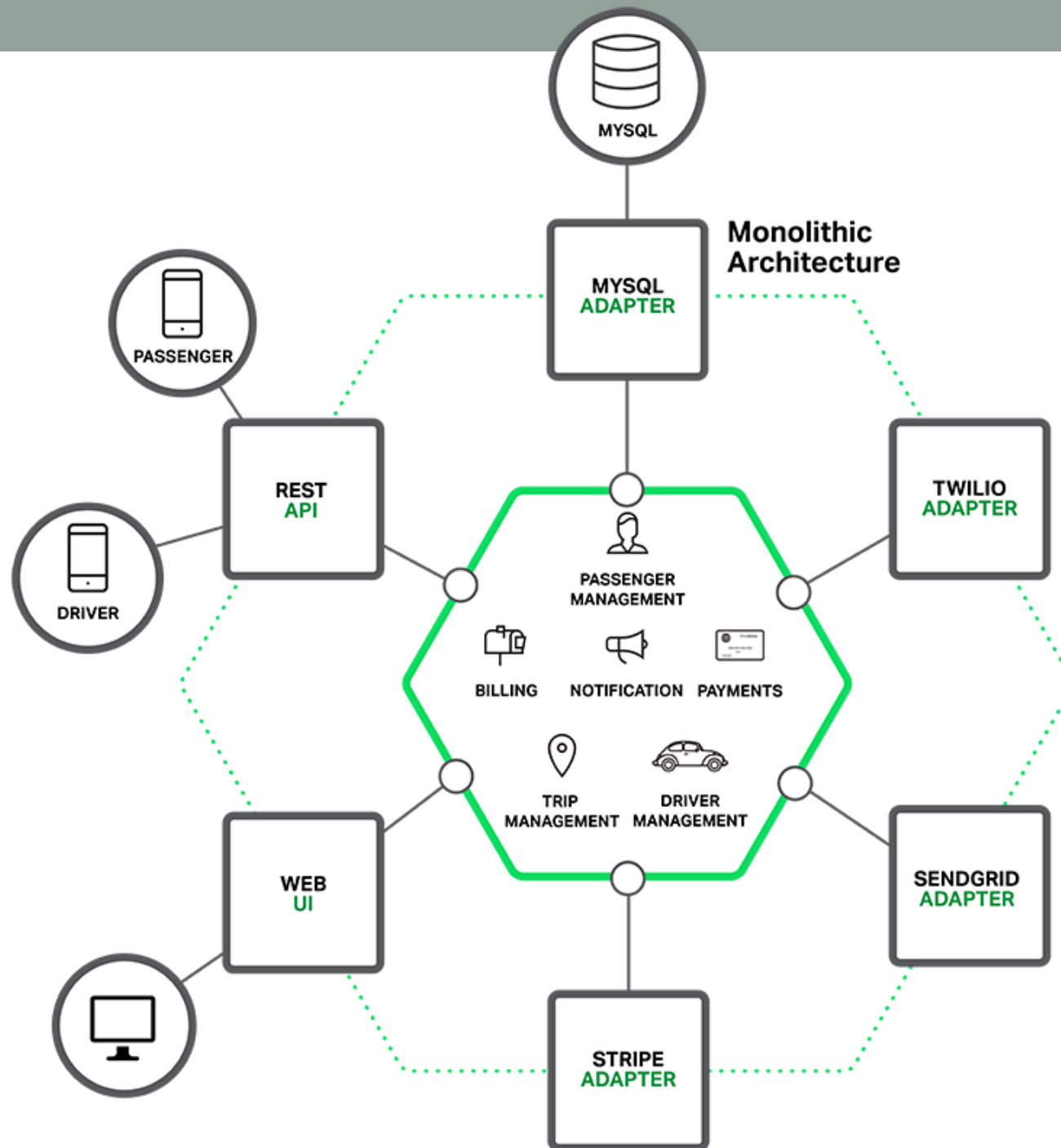
Supporting functions

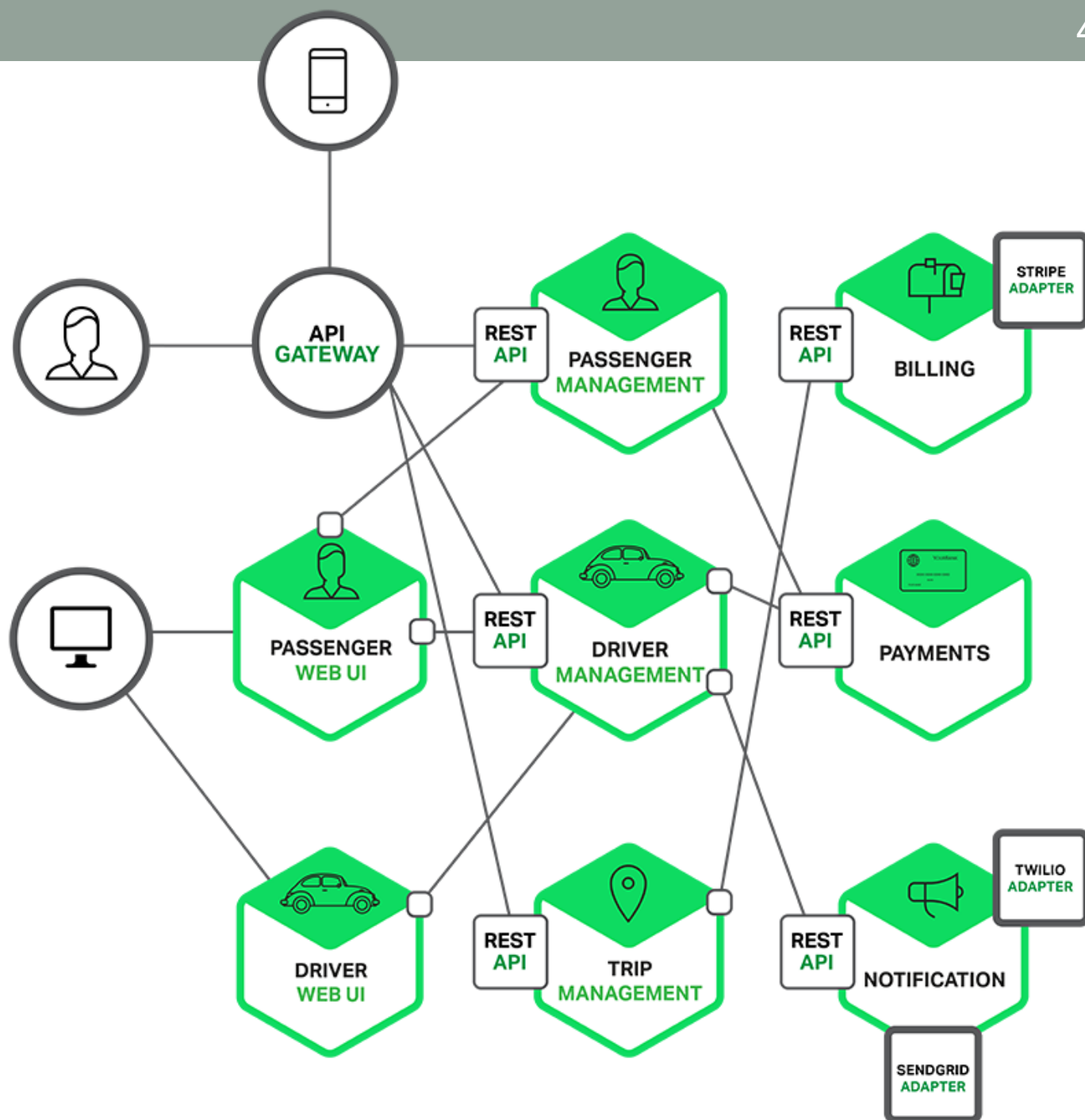
Check password validity
Delete password
Backup password database
Recover password database
Check database integrity
Repair password DB

Exercício

- Como você faria a decomposição em microsserviços do sistema descrito na imagem a seguir?

<https://www.nginx.com/blog/introduction-to-microservices/>

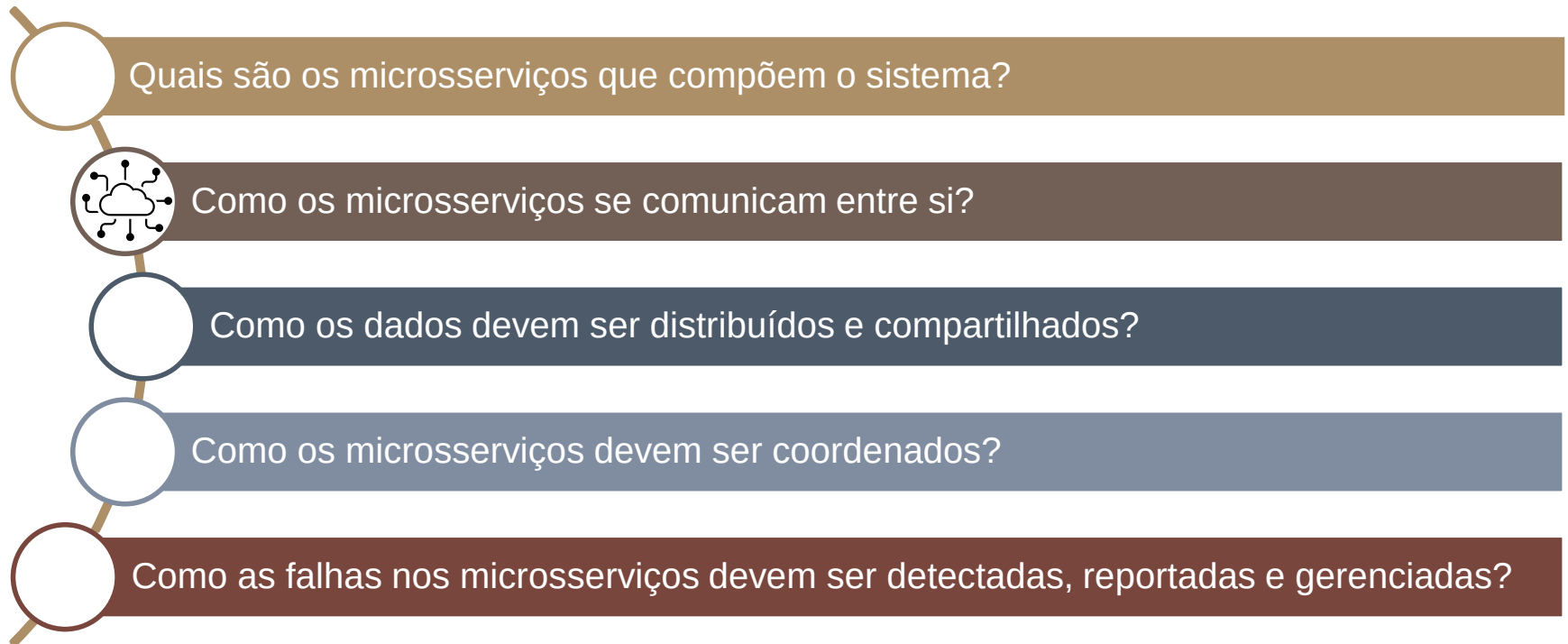




MICROSSERVIÇOS

Comunicação

Decisões Arquiteturais



Comunicação

- Os microsserviços se comunicam trocando mensagens.
- Uma mensagem enviada entre serviços inclui algumas informações administrativas, uma solicitação de serviço e os dados necessários para entregar o serviço solicitado.
- Os serviços retornam uma resposta às mensagens de solicitação de serviço.
- Exemplo:
 - Um serviço de autenticação pode enviar uma mensagem para um serviço de login que inclui o nome inserido pelo usuário.
 - A resposta pode ser um token associado a um nome de usuário válido ou pode ser um erro informando que não há usuário registrado.

Comunicação

- Os serviços se comunicam trocando mensagens que incluem informações sobre o originador da mensagem, bem como os dados que são a entrada ou saída da solicitação.
- Ao projetar uma arquitetura de microsserviços, você precisa estabelecer um padrão de comunicação que os microsserviços devem seguir.

Decisões Arquiteturais

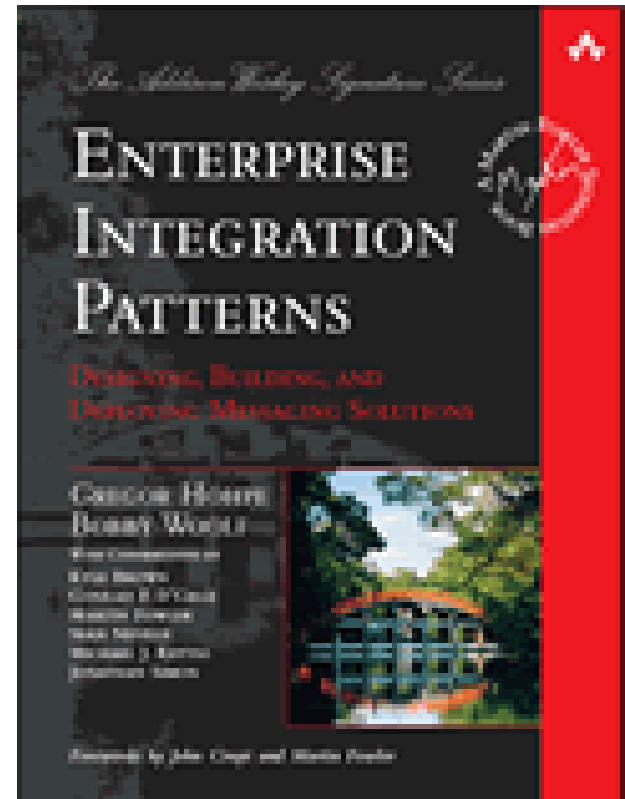
A interação entre microsserviços deve ser síncrona ou assíncrona?

Os microsserviços devem se comunicar diretamente ou por meio do *middleware* do *broker* de mensagens?

Qual protocolo deve ser usado para mensagens trocadas entre os microsserviços?

Decisões Arquiteturais

- Catálogos de padrões:
 - <https://www.enterpriseintegrationpatterns.com/>

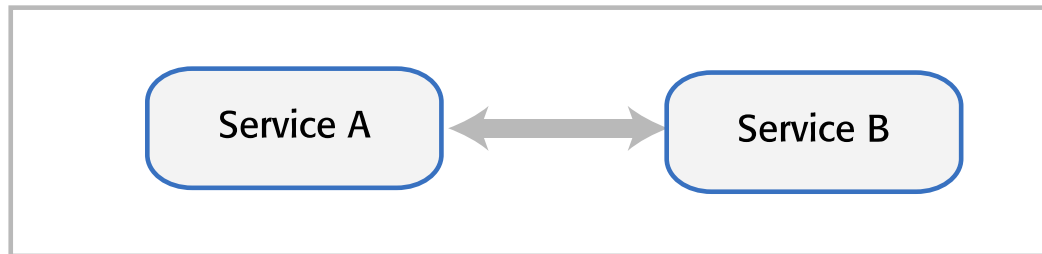


Comunicação direta e indireta

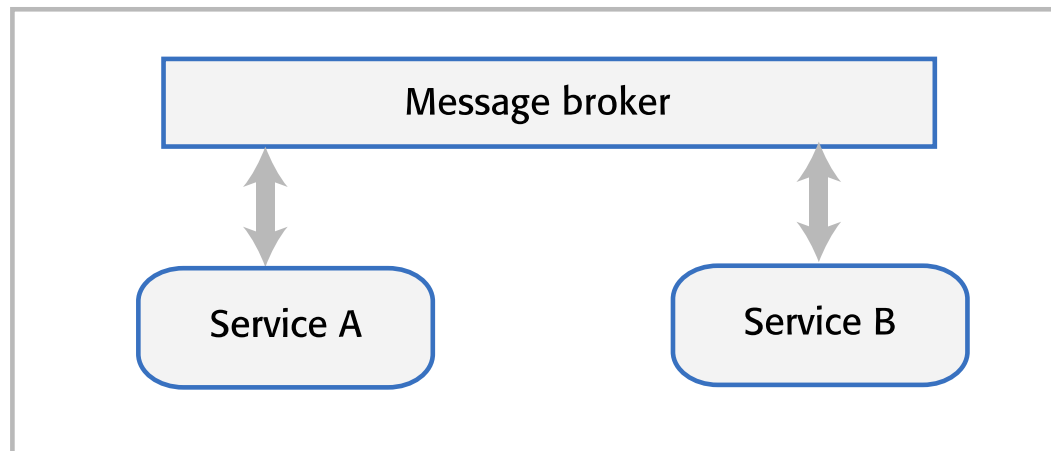
- A **comunicação direta** de serviço requer que os serviços que interagem conheçam o endereço um do outro.
- Os serviços interagem enviando solicitações diretamente para esses endereços.
- A **comunicação indireta** envolve nomear o serviço necessário e enviar essa solicitação para um agente de mensagens (às vezes chamado de barramento de mensagens).
- O agente de mensagens é então responsável por localizar o serviço que pode atender à solicitação de serviço.

Comunicação direta e indireta

Direct communication - A and B send messages to each other



Indirect communication - A and B communicate through a message broker

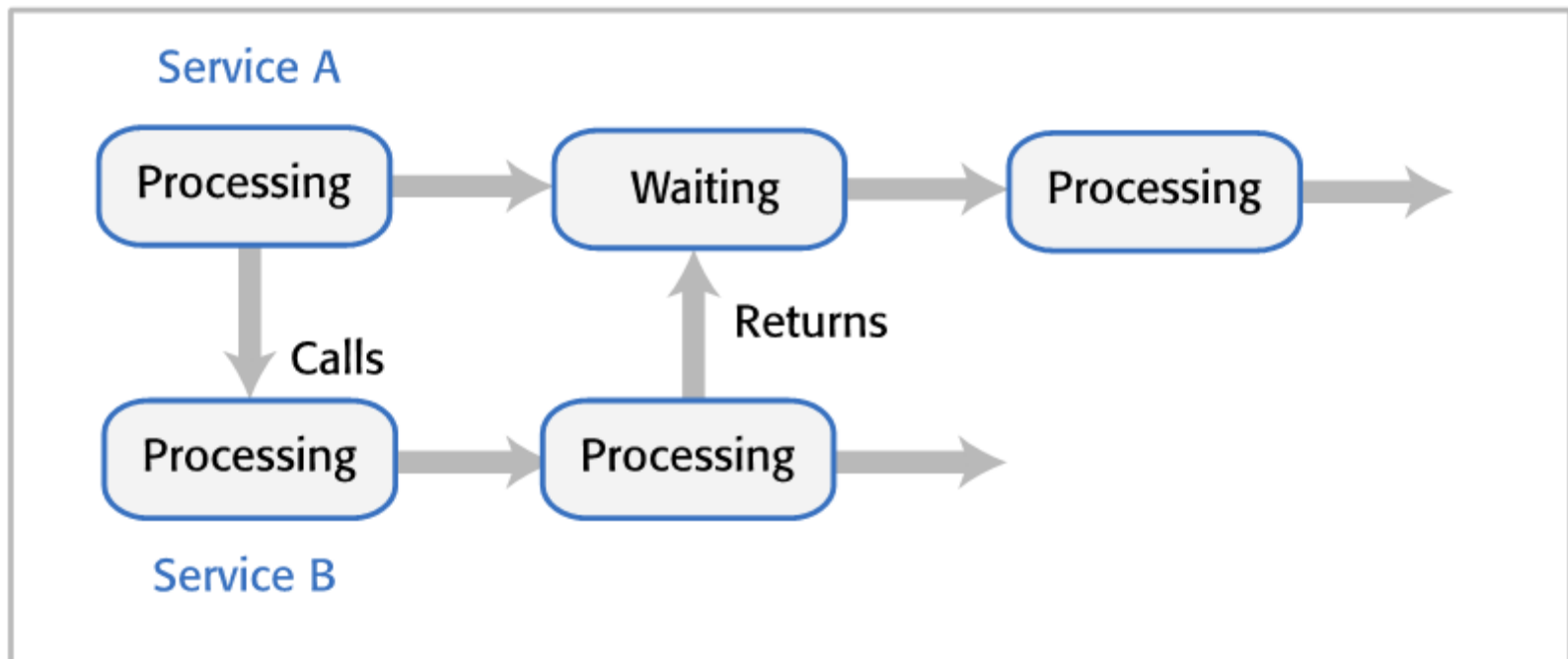


Comunicação síncrona e assíncrona

- Em uma **interação síncrona**, o serviço A emite uma solicitação ao serviço B.
- O serviço A então suspende o processamento enquanto B está processando a solicitação.
- Ele espera até que o serviço B retorne as informações necessárias antes de continuar a execução.

Comunicação síncrona e assíncrona

Synchronous - A waits for B

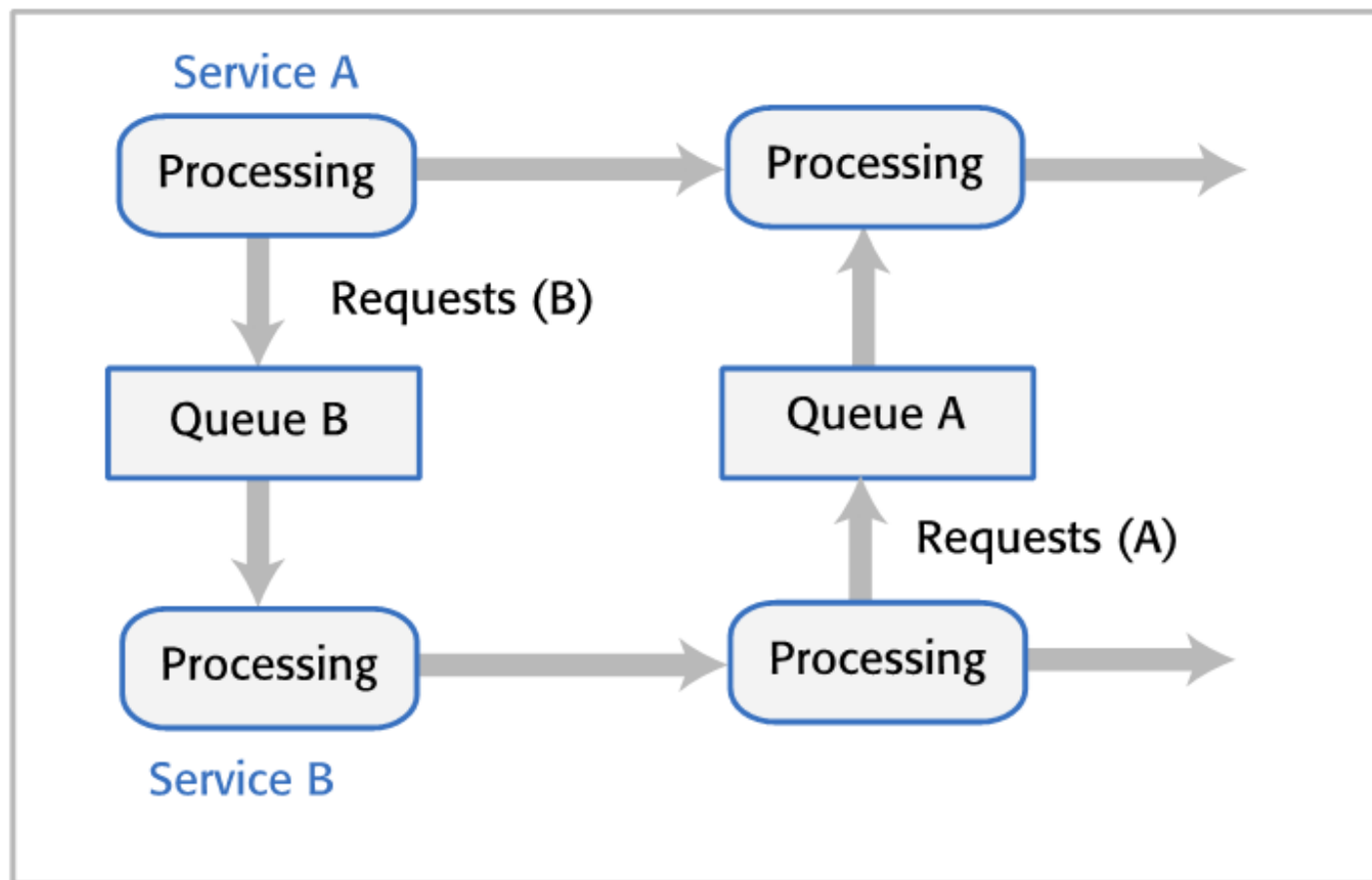


Comunicação síncrona e assíncrona

- Em uma **interação assíncrona**, o serviço A emite a solicitação que é enfileirada para processamento pelo serviço B.
- A então continua o processamento sem esperar que B termine seus cálculos.
- Algum tempo depois, o serviço B conclui a solicitação anterior do serviço A e enfileira o resultado a ser recuperado por A.
- O serviço A, portanto, precisa verificar sua fila periodicamente para ver se um resultado está disponível.

Comunicação síncrona e assíncrona

Asynchronous - A and B execute concurrently

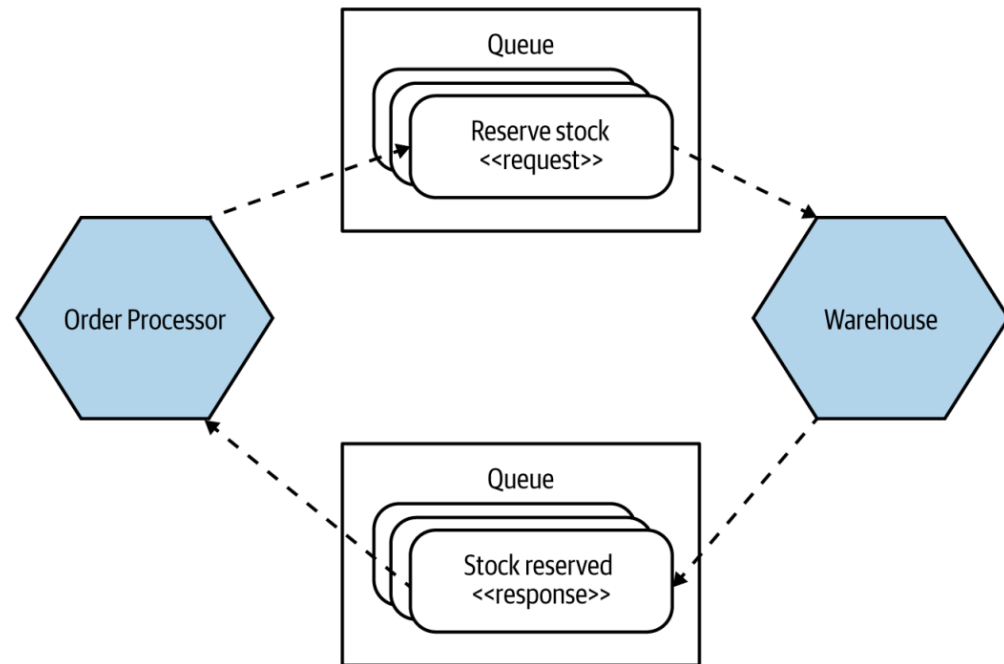


Comunicação assíncrona

- Vantagens:
 - Diminuição do acoplamento entre os microsserviços.
 - Diminui a contenção de microsserviços que aguardam por tarefas longas.
- Desvantagens:
 - Complexidade adicional do modelo assíncrono.
 - Grande variedade de opções de implementação.

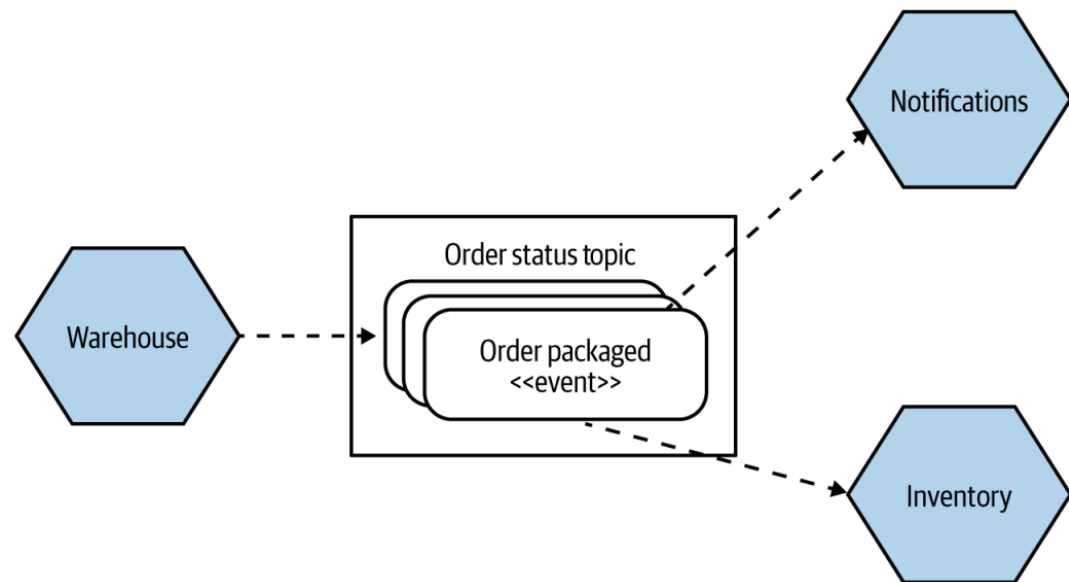
Comunicação assíncrona

- Exemplo: baseada em mensagens de requisição/resposta



Comunicação assíncrona

- Exemplo: baseada em eventos



MICROSSERVIÇOS

Distribuição

Decisões Arquiteturais

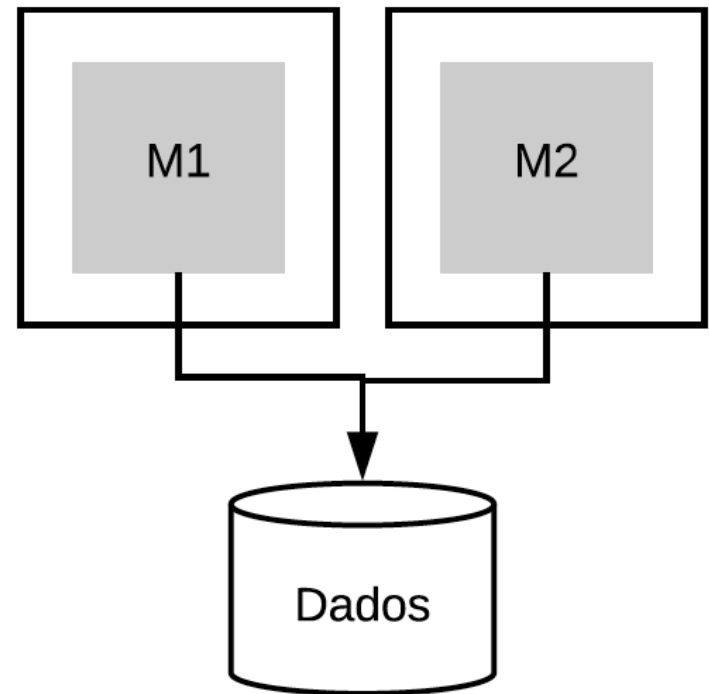
- Quais são os microsserviços que compõem o sistema?
- Como os microsserviços se comunicam entre si?
-  Como os dados devem ser distribuídos e compartilhados?
- Como os microsserviços devem ser coordenados?
- Como as falhas nos microsserviços devem ser detectadas, reportadas e gerenciadas?

Projeto de dados

- Você deve isolar os dados em cada serviço do sistema com o mínimo de compartilhamento de dados possível.
- Se o compartilhamento de dados for inevitável, você deve projetar microsserviços para que a maioria dos compartilhamentos seja 'somente leitura', com um número mínimo de serviços responsáveis pelas atualizações de dados.
- Se os serviços forem replicados em seu sistema, você deverá incluir um mecanismo que possa manter as cópias do banco de dados usadas pelos serviços de réplica consistentes.

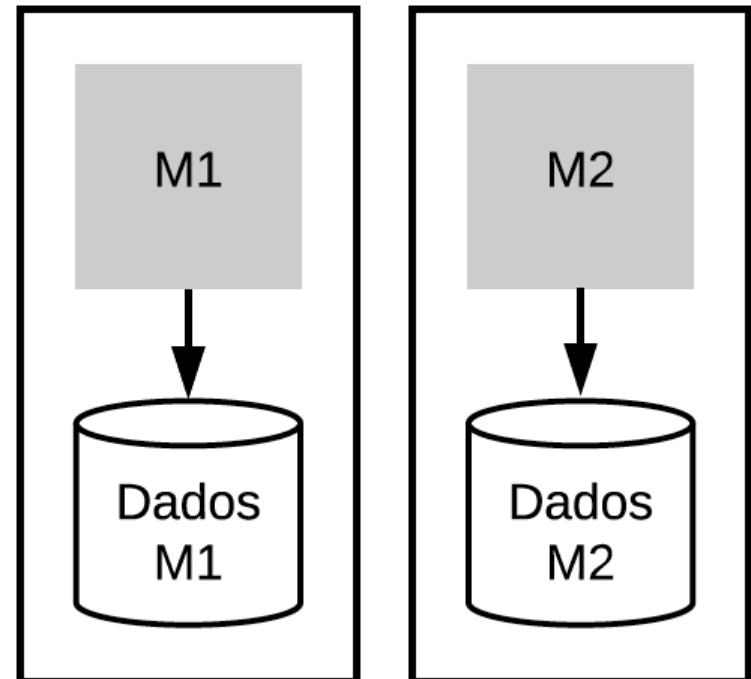
Projeto de Dados

- Compartilhamento de dados não é recomendado como solução geral
 - Apenas em casos particulares
- Motivo:
 - Aumenta acoplamento entre M1 e M2



Projeto de Dados

- Arquitetura recomendada
- Motivo:
 - Não existe acoplamento de dados entre M1 e M2
 - Logo, M1 e M2 podem evoluir de modo independente



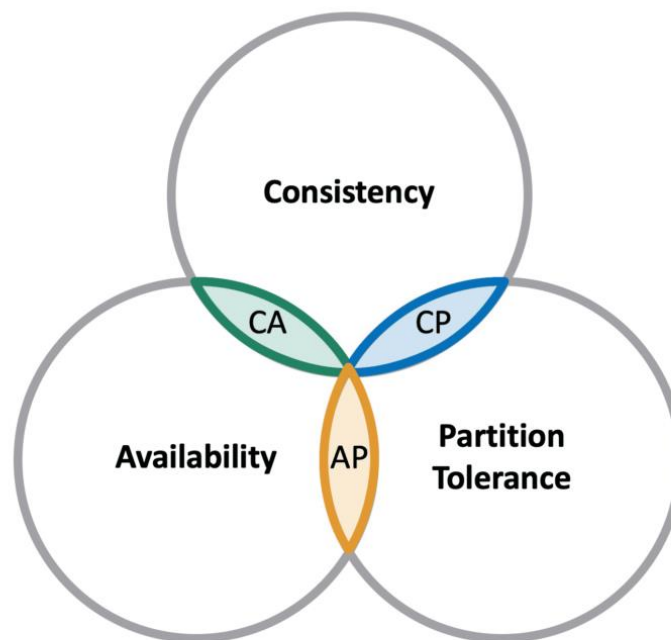
Teorema CAP

- Teorema CAP trata de cenários envolvendo bancos de dados distribuídos.
- Existirão partições (P, partition) entre os bancos de dados.
 - Ex.: suponha dois bancos de dados A e B, localizados em seus respectivos microsserviços. Logo, um problema de rede, pode fazer com que A não consiga se comunicar com B e vice-versa.
- O arquiteto do sistema tem que fazer uma escolha:
 - Priorizar disponibilidade (A, availability), abrindo mão da consistência (C, consistency), pelo menos em certos casos.
 - Priorizar consistência (C) e, abrir mão, quando inevitável, da disponibilidade (D).
- Portanto, sistemas com dados distribuídos são AP ou CP, mas nunca CAP.

Teorema CAP

- Priorizar disponibilidade (A), abrindo mão da consistência (C).
 - Suponha que a versão mais nova de um dado está em A, mas uma partição na rede impede que ela seja propagada para B. Como o primeiro objetivo é disponibilidade, o sistema vai entregar para os clientes de B a versão desatualizada do dado.
- Priorizar consistência (C) e, abrir mão da disponibilidade (D).
 - O motivo é que para garantir consistência temos que adotar um protocolo (ou algoritmo) para propagar dados de A para B e vice-versa. Isto é, esse protocolo vai manter os dois BDs sincronizados. Mas, para isso, ele tem que travar um BD quando o outro não estiver disponível. Consequentemente, o BD travado não vai responder requisições de seus clientes para não entregar dados desatualizados ou inconsistentes.

Teorema CAP



Gerenciamento de inconsistências

- Uma transação ACID agrupa um conjunto de atualizações de dados em uma única unidade para que todas as atualizações sejam concluídas ou nenhuma delas seja.
- As transações ACID são “limitantes” em uma arquitetura de microsserviços.
- Os bancos de dados usados por diferentes microsserviços ou réplicas de microsserviços não precisam ser completamente consistentes o tempo todo.

Gerenciamento de inconsistências

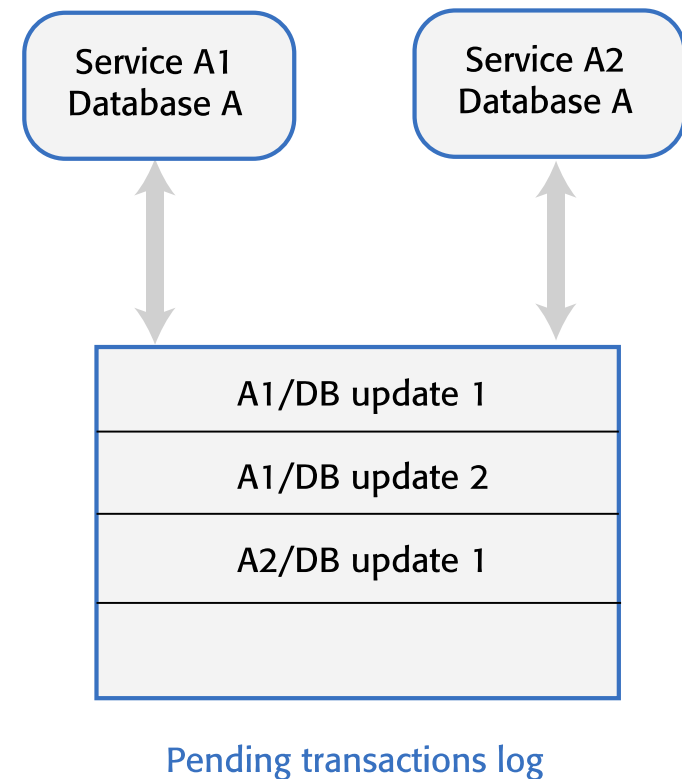
- Inconsistência de dados dependentes:
 - As ações ou falhas de um serviço podem fazer com que os dados gerenciados por outro serviço se tornem inconsistentes.
- Inconsistência da réplica:
 - Existem várias réplicas do mesmo serviço que estão sendo executadas simultaneamente. Todos eles têm sua própria cópia do banco de dados e cada um atualiza sua própria cópia dos dados do serviço. Você precisa de uma maneira de tornar esses bancos de dados “eventualmente consistentes” para que todas as réplicas trabalhem nos mesmos dados.

Consistência eventual

- A consistência eventual é uma situação em que o sistema garante que os bancos de dados eventualmente se tornarão consistentes.
- Você pode implementar consistência eventual mantendo um log de transações.
- Quando uma alteração no banco de dados é feita, isso é registrado em um log de 'atualizações pendentes'.
- Outras instâncias de serviço examinam esse log, atualizam seu próprio banco de dados e indicam que fizeram a alteração.

Consistência eventual

- Usando um log de transações pendentes:



MICROSSERVIÇOS

Coordenação

Decisões Arquiteturais

- Quais são os microsserviços que compõem o sistema?
- Como os microsserviços se comunicam entre si?
- Como os dados devem ser distribuídos e compartilhados?
-  Como os microsserviços devem ser coordenados?
- Como as falhas nos microsserviços devem ser detectadas, reportadas e gerenciadas?

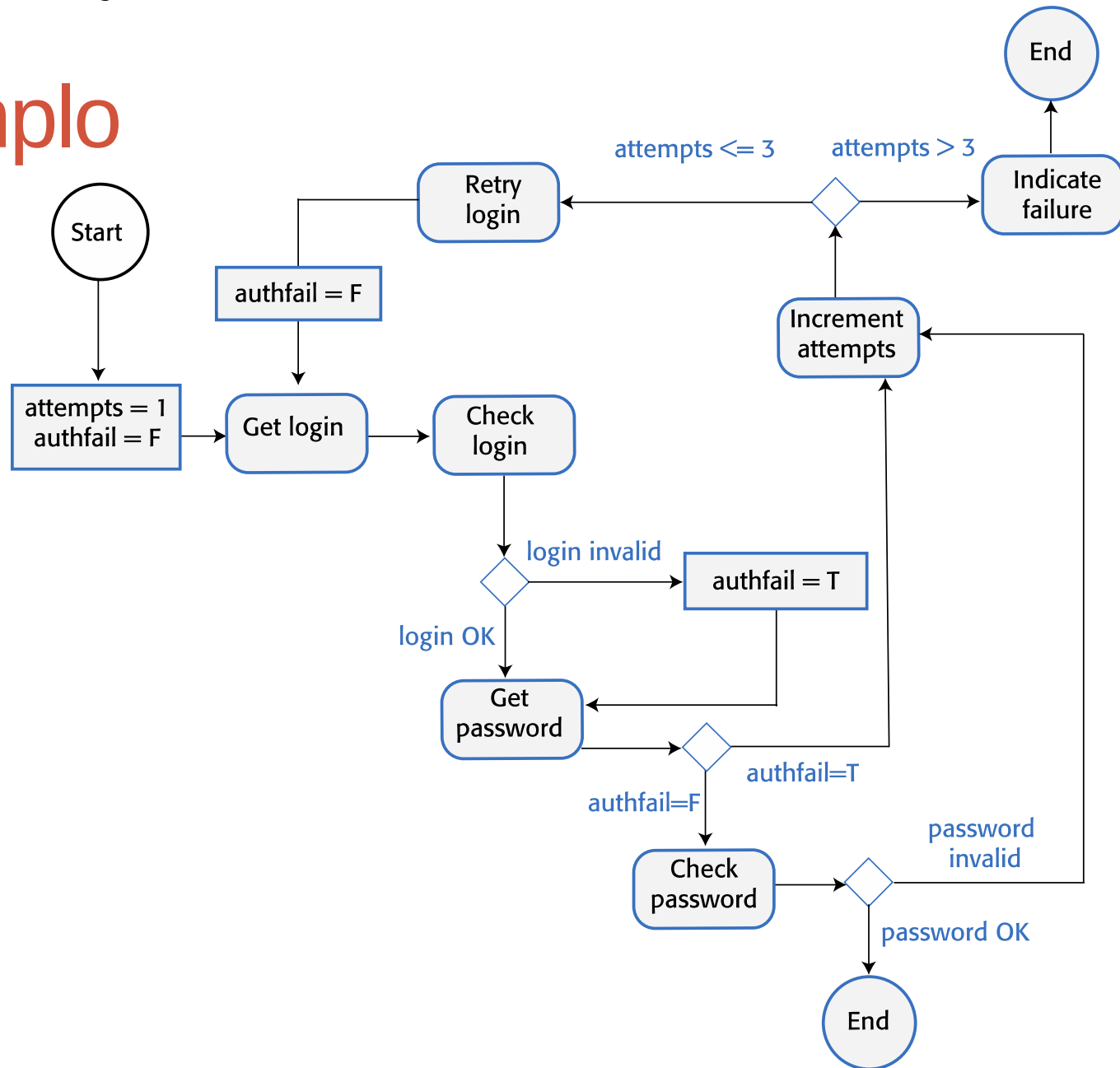
Coordenação

- Certos estilos de interação com microsserviços envolvem uma série de interações nas quais as operações devem ser realizadas em uma ordem específica.
- Isso é chamado de fluxo de trabalho e demanda um certo tipo de coordenação.

Exemplo

- Um fluxo de trabalho de autenticação para autenticação de UID/senha mostra as etapas envolvidas na autenticação de um usuário.
- Neste exemplo, o usuário tem 3 tentativas de login antes que o sistema indique que o login falhou.

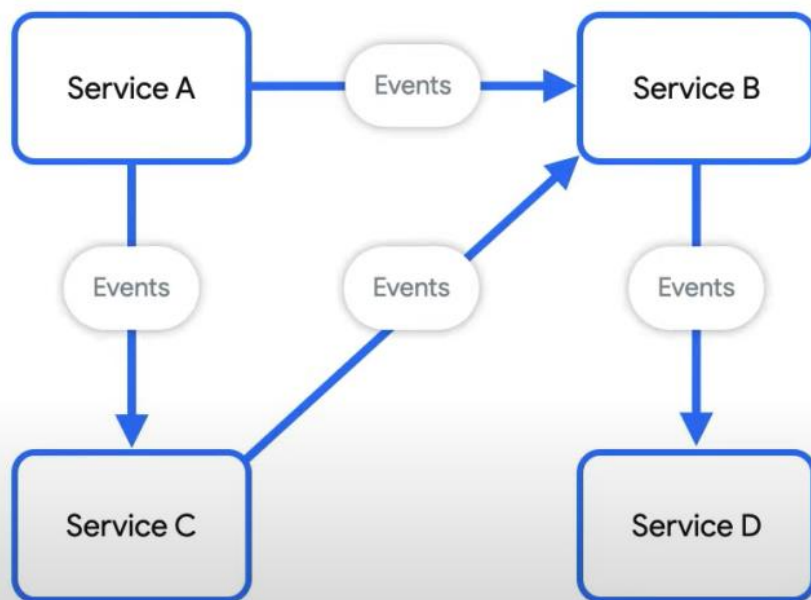
Exemplo



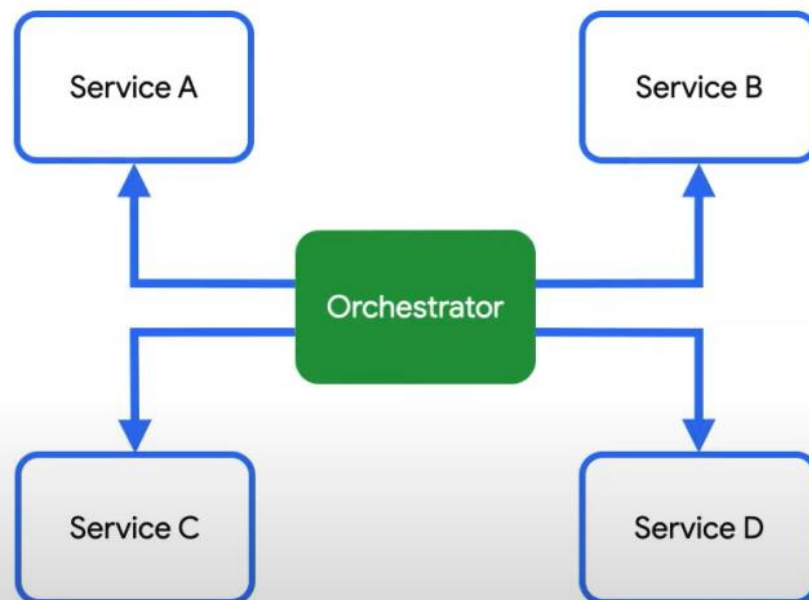
Orquestração

- Frequentemente, uma operação de negócio deve ser executada em múltiplos serviços ou microsserviços.
- Nesse contexto, chama-se de **orquestrador** o módulo que dispara essa execução e monitora os seus resultados.
- Representa um fluxo de trabalho centralizado.

Choreography



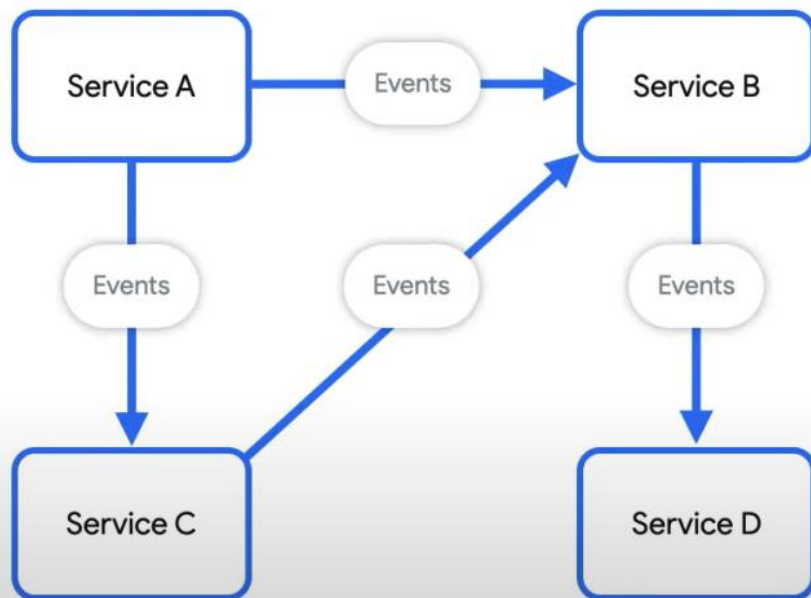
Orchestration



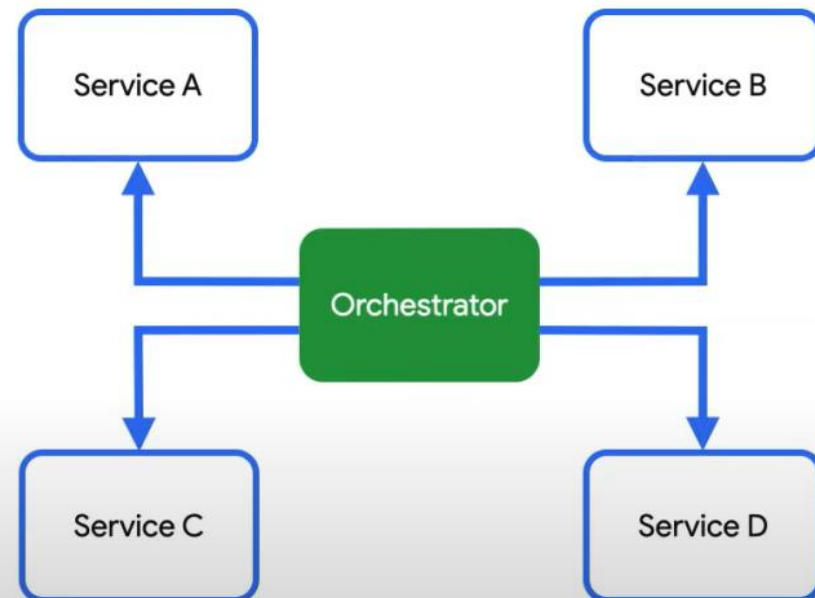
Coreografia

- Frequentemente, uma operação de negócio deve ser executada em múltiplos serviços ou microsserviços.
- Nesse contexto, não existe um componente responsável pelo controle do processo, mas uma troca de eventos, e cada microsserviço é responsável por observar e responder a eventos de seu interesse de forma autônoma.
- Representa um fluxo de trabalho descentralizado.

Choreography



Orchestration



MICROSSERVIÇOS

Observação

Decisões Arquiteturais

- Quais são os microsserviços que compõem o sistema?
- Como os microsserviços se comunicam entre si?
- Como os dados devem ser distribuídos e compartilhados?
- Como os microsserviços devem ser coordenados?
- Como as falhas nos microsserviços devem ser detectadas, reportadas e gerenciadas?

Tipos de falhas

- Falha de serviço interno
 - Essas são condições detectadas pelo serviço e podem ser relatadas ao cliente de serviço em uma mensagem de erro. Um exemplo desse tipo de falha é um serviço que usa uma URL como entrada e descobre que esse é um link inválido.
- Falha de serviço externo
 - Essas falhas têm uma causa externa, o que afeta a disponibilidade de um serviço. A falha pode fazer com que o serviço não responda e ações devem ser tomadas para reiniciar o serviço.
- Falha de performance
 - O desempenho do serviço degrada-se a um nível inaceitável. Isso pode ser devido a uma carga pesada ou um problema interno com o serviço. O monitoramento de serviços externos pode ser usado para detectar falhas de desempenho e serviços sem resposta.

Observabilidade

- Observabilidade é a capacidade de entender o estado interno de um sistema, analisando sinais externos produzidos pelo mesmo.
- Problema: aplicações executando na nuvem, de forma distribuída e em infraestruturas cada vez mais complexas e automatizadas.
- Importante para identificar, diagnosticar e resolver problemas rapidamente.
- Auxilia a aumentar a eficiência, confiabilidade e segurança de sistemas.

Pilares da observabilidade

- Métricas: Indicadores quantitativos que representam o estado e o desempenho do sistema. Exemplos incluem latência, vazão e taxa de erros.
- Logs: Registros detalhados de eventos e interações do sistema, auxiliando na identificação de problemas.
- Rastreamento: Permite acompanhar o caminho que uma requisição percorre pelos diferentes componentes de um sistema (distribuído).

MICROSSERVIÇOS

Resumindo...

Pontos Chave

- Um microserviço é um componente de software independente e autocontido que é executado em seu próprio processo e se comunica com outros microserviços usando protocolos leves.
- Os microserviços em um sistema podem ser implementados usando diferentes linguagens de programação e tecnologias de banco de dados.
- Os microserviços têm uma única responsabilidade e devem ser projetados para que possam ser facilmente alterados sem a necessidade de alterar outros microserviços no sistema.

Pontos Chave

- A arquitetura de microsserviços é um estilo de arquitetura no qual o sistema é construído a partir de microsserviços de comunicação. É adequado para sistemas baseados em nuvem, onde cada microsserviço pode ser executado em seu próprio contêiner.
- As duas responsabilidades mais importantes dos arquitetos de um sistema de microsserviços são decidir como estruturar o sistema em microsserviços e decidir como os microsserviços devem se comunicar e ser coordenados.
- As decisões de comunicação e coordenação incluem decidir sobre protocolos de comunicação de microsserviços, compartilhamento de dados, se os serviços devem ser coordenados centralmente e gerenciamento de falhas.